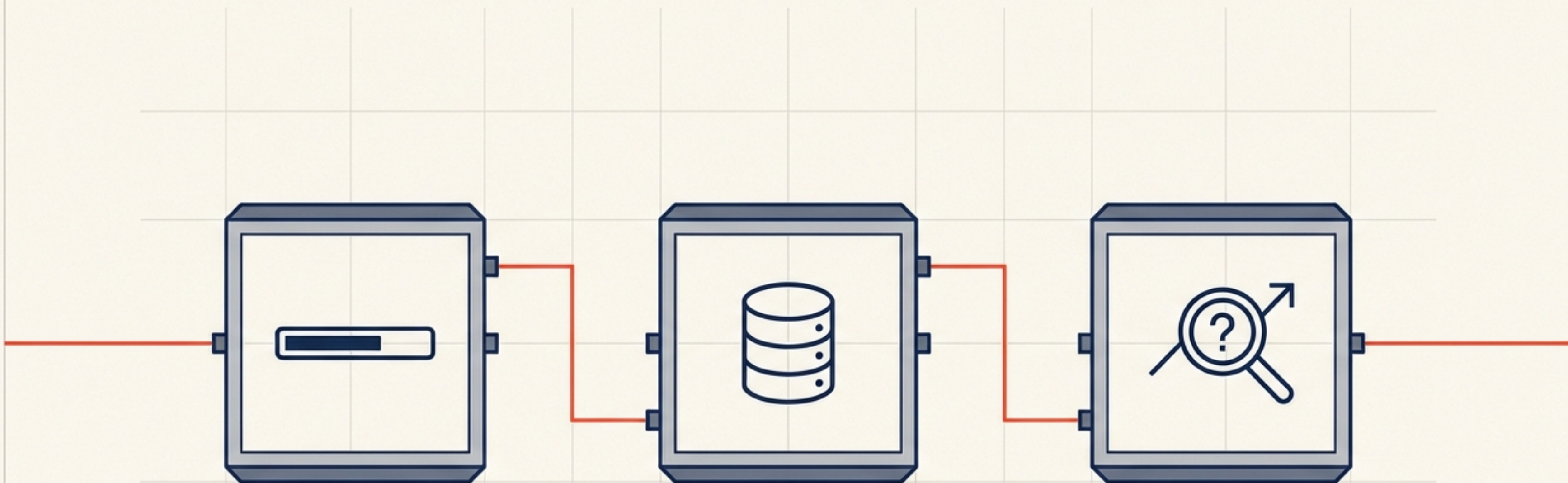


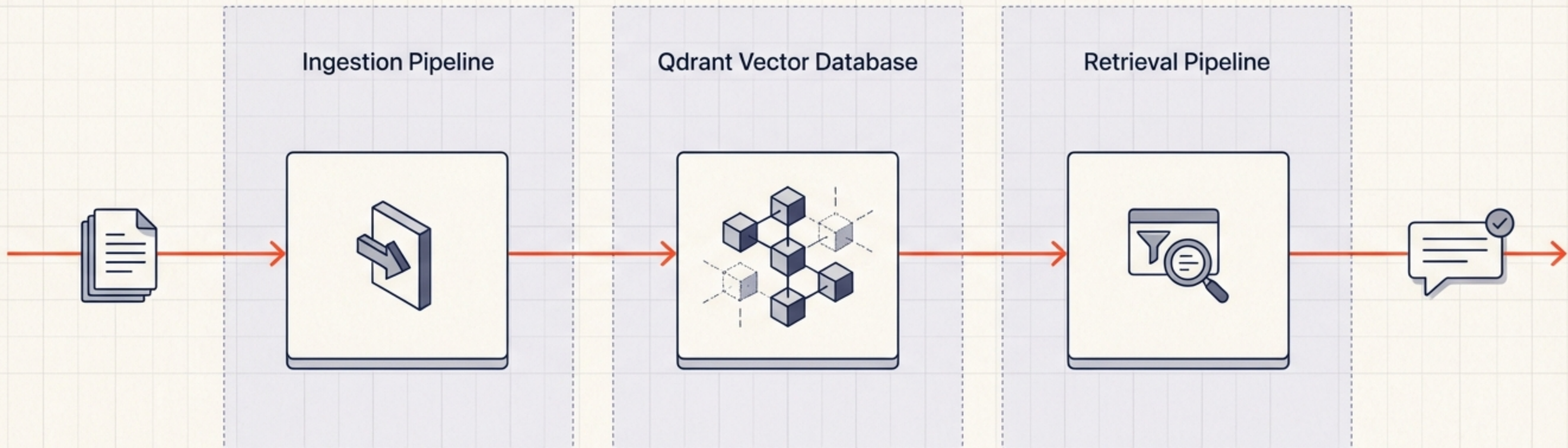


# RAGWire

The Production-Grade RAG Toolkit.

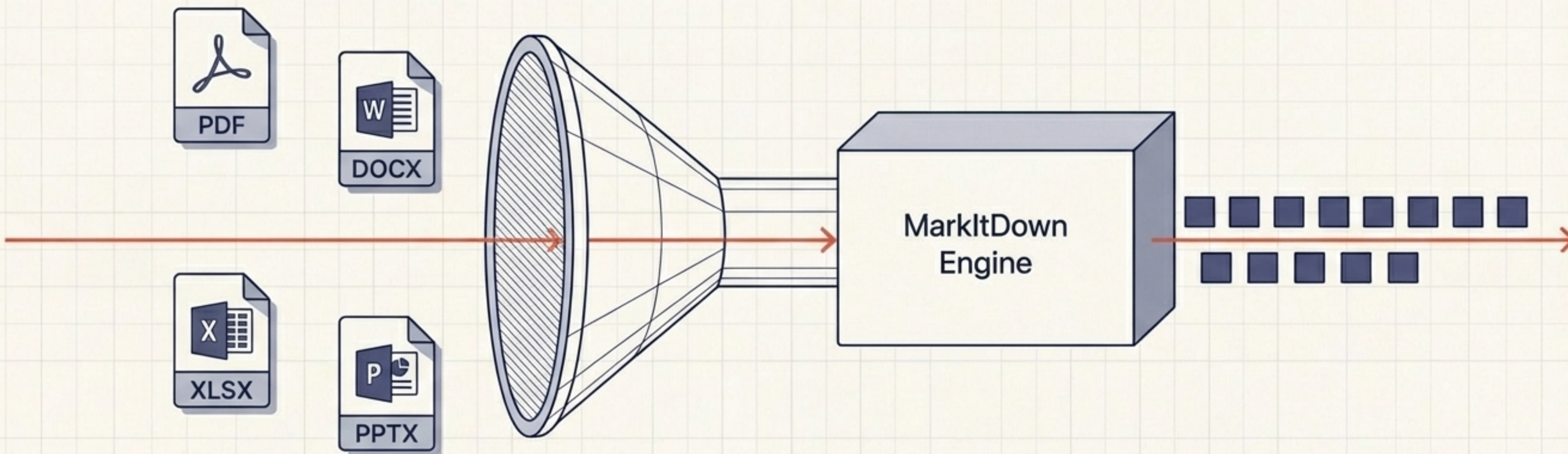
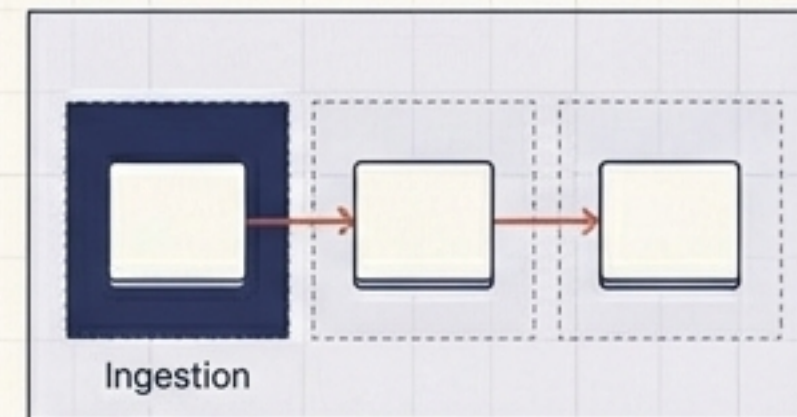
# End-to-End Pipeline Automation

RAGWire handles the full Retrieval-Augmented Generation lifecycle—from raw documents to advanced retrieval—so developers can focus on application logic rather than plumbing.



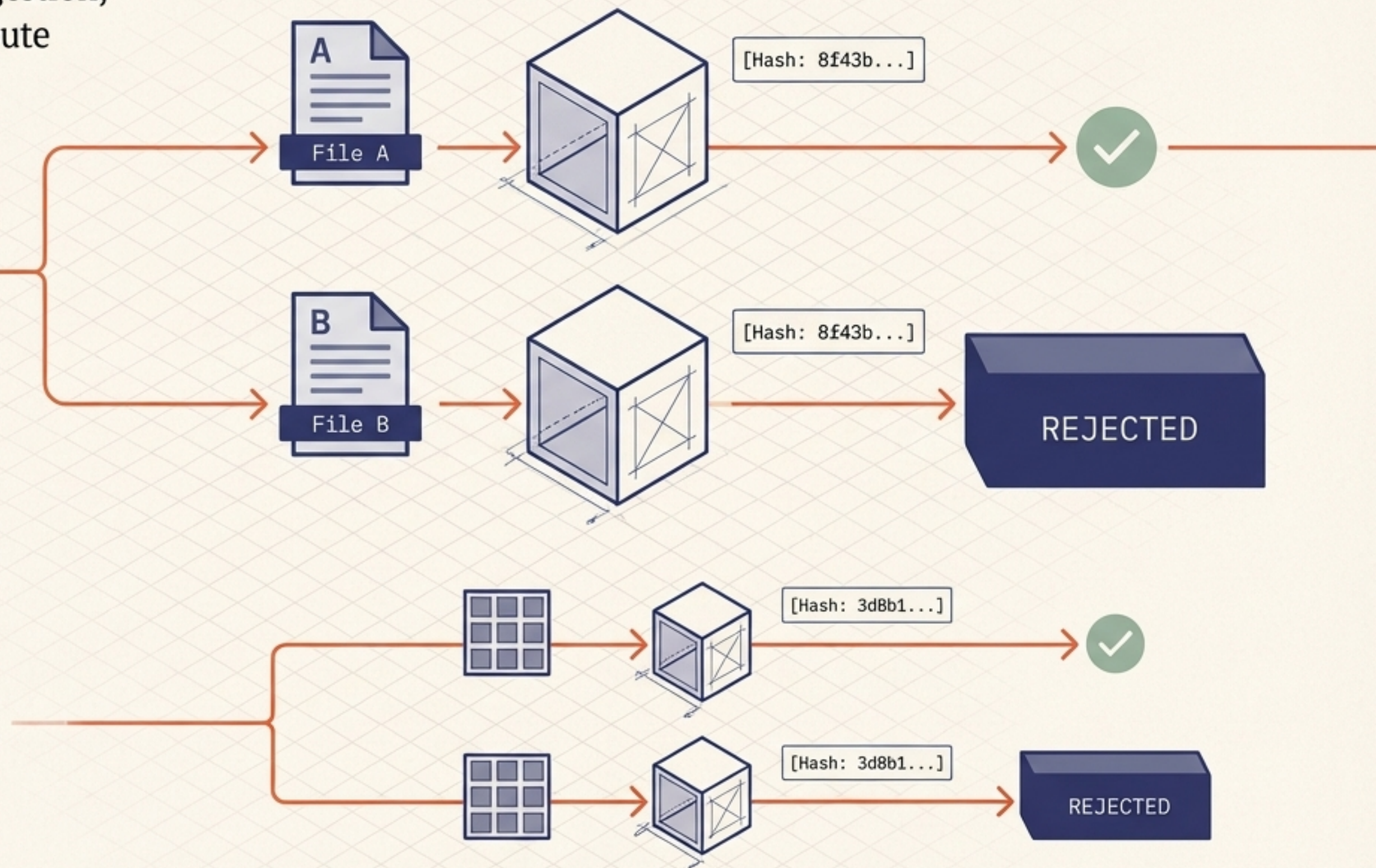
# Universal Document Loading

Seamlessly parse complex, unstructured documents into clean, structured formats using native Markdown integration.



# Zero-Redundancy Architecture

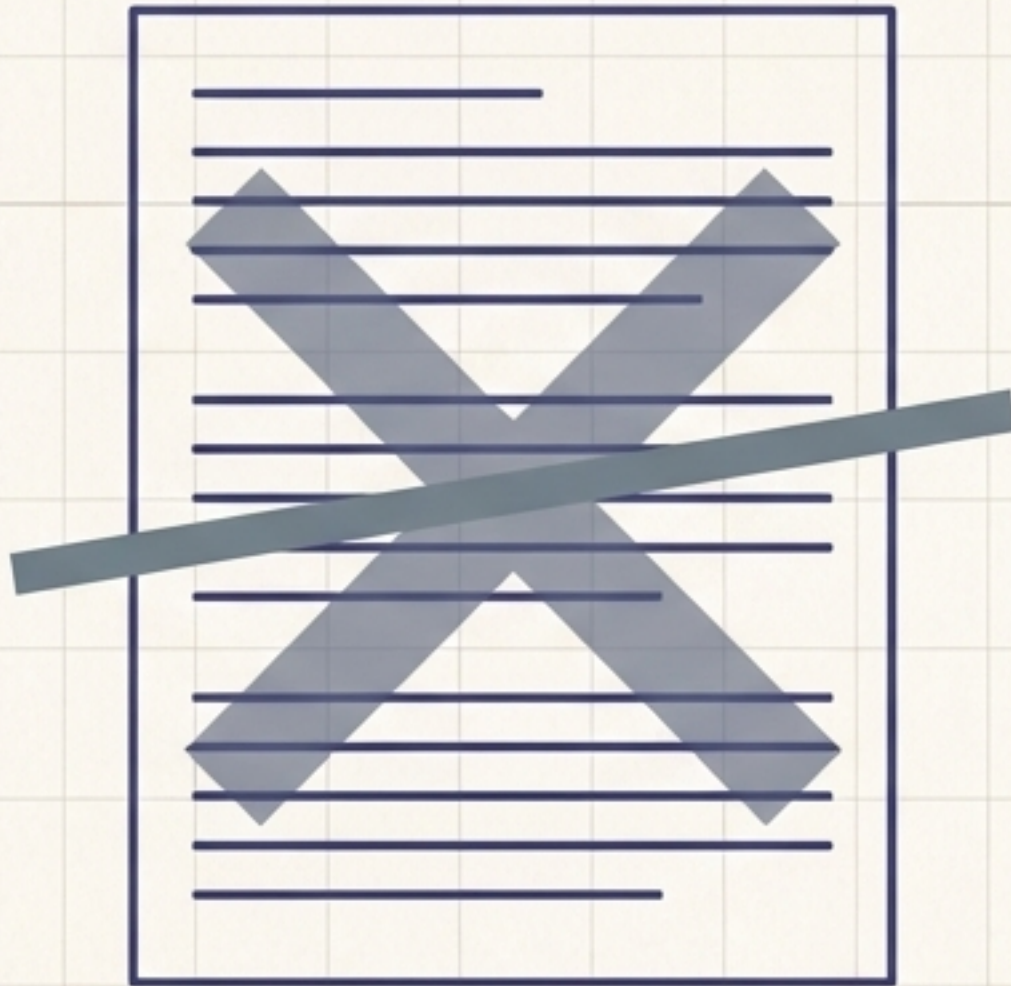
Built-in SHA256 hashing at both the file and chunk levels ensures zero duplicate ingestion, ingestion, eliminating redundant compute and vector storage costs.



# Context-Preserving Text Splitting

Markdown-aware and recursive chunking strategies ensure semantic meaning is never fractured across chunk boundaries.

Naive Splitting



RAGWire Splitting



# Automated Context Enrichment

Automatically extract and append LLM-generated metadata to chunks, unlocking highly precise pre-filtering and exact-match queries downstream.

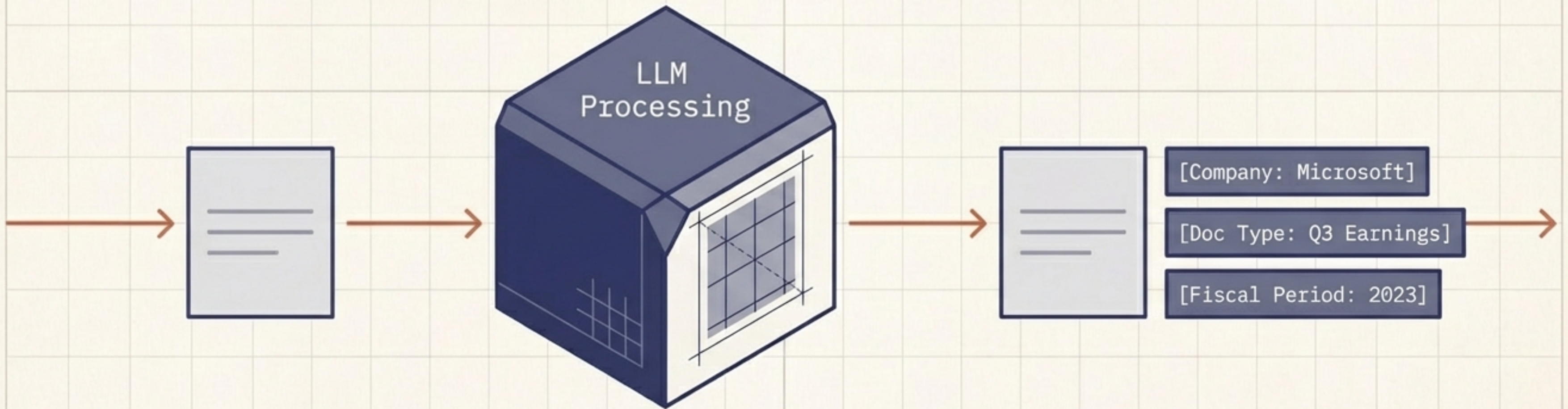
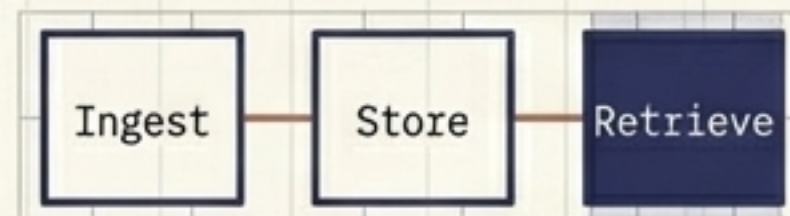


Diagram illustrating a data hazard in a 5-stage pipeline (IF, DE, EX, ST, WB). The instruction sequence is:

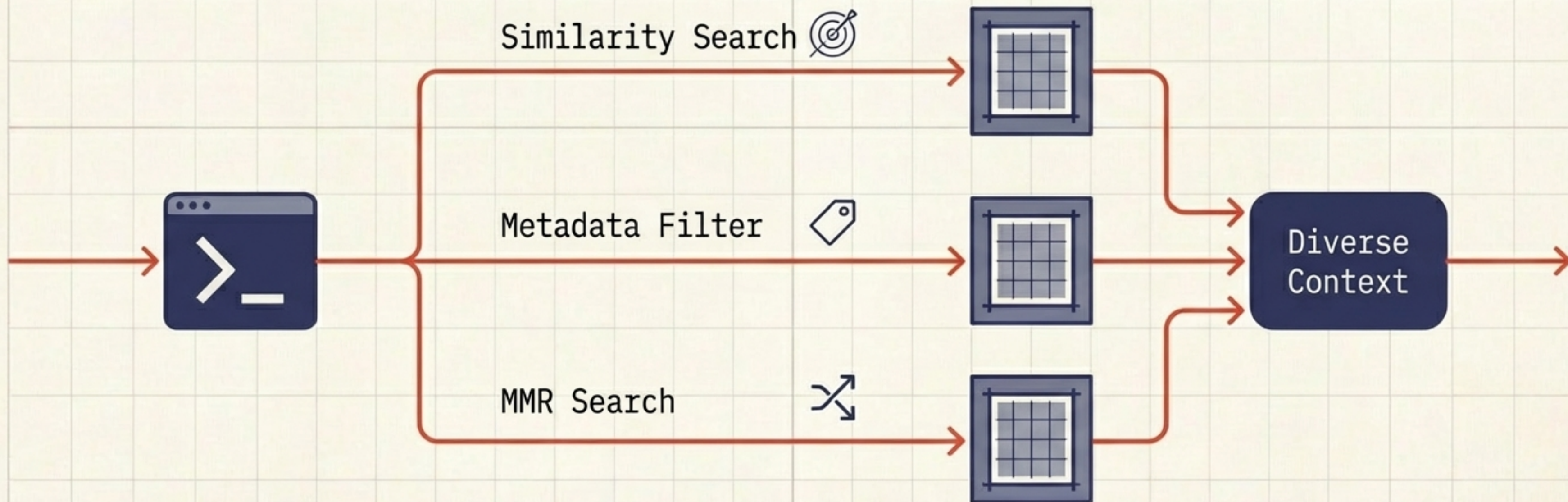
- Instruction 1: `add $t0, $t0, $t1` (Stages: IF, DE, EX, ST, WB)
- Instruction 2: `add $t2, $t0, $t3` (Stages: IF, DE, EX, ST, WB)
- Instruction 3: `add $t4, $t2, $t5` (Stages: IF, DE, EX, ST, WB)

A data hazard exists between Instruction 1 and Instruction 2. The hazard is detected in the DE stage of Instruction 2, and the pipeline stalls for 2 cycles (indicated by a red 'X' and a red '2' in the hazard detection stage).

# Advanced Retrieval Mechanics



Move beyond basic cosine similarity. Leverage Hybrid Search and MMR to retrieve diverse, highly relevant context windows without result homogenization.



# Provider Agnostic Infrastructure

Seamlessly swap between local models for strict privacy and cloud models for raw processing power.

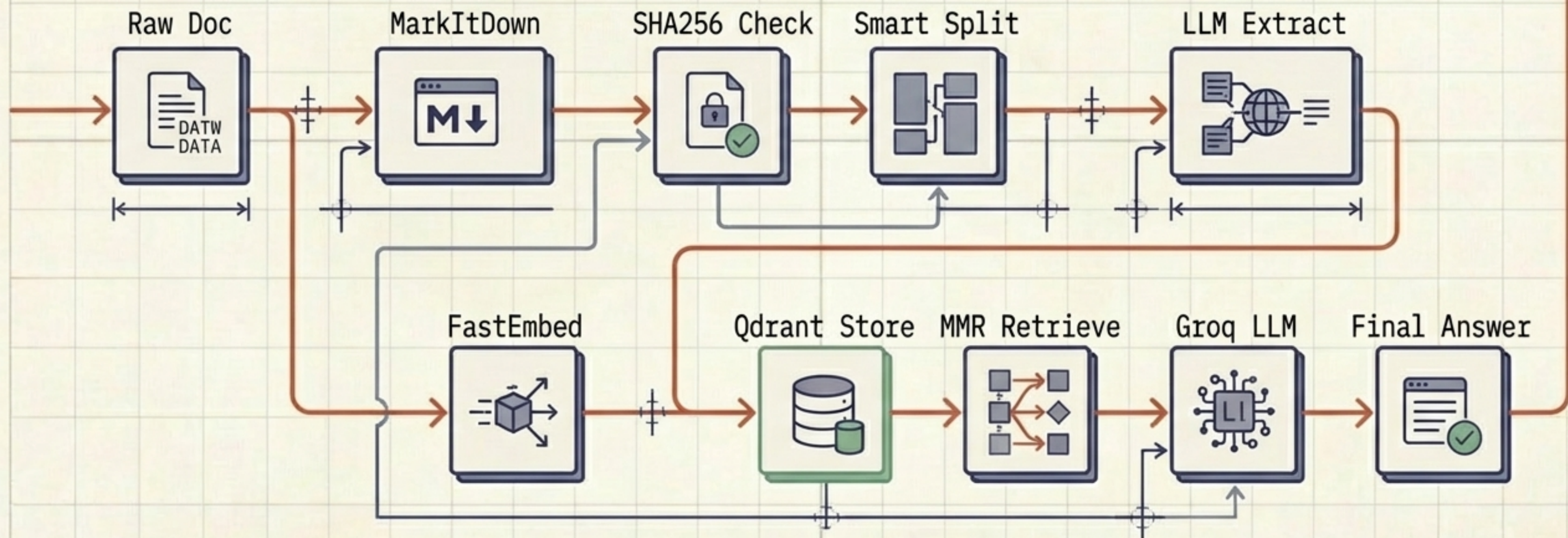


The diagram illustrates a provider-agnostic infrastructure. A central horizontal line on the left has a bracket that branches into seven arrows pointing to the 'Provider' column of the table. Similarly, a central horizontal line on the right has a bracket that branches into seven arrows pointing to the 'Access Tier' column. The table lists seven providers: Ollama, OpenAI, Google Gemini, Groq, Anthropic, HuggingFace, and FastEmbed. Each provider row has a green checkmark in the 'Embeddings' column, a green checkmark or a grey dash in the 'LLM' column, and an icon representing the access tier (local device or cloud) in the 'Access Tier' column.

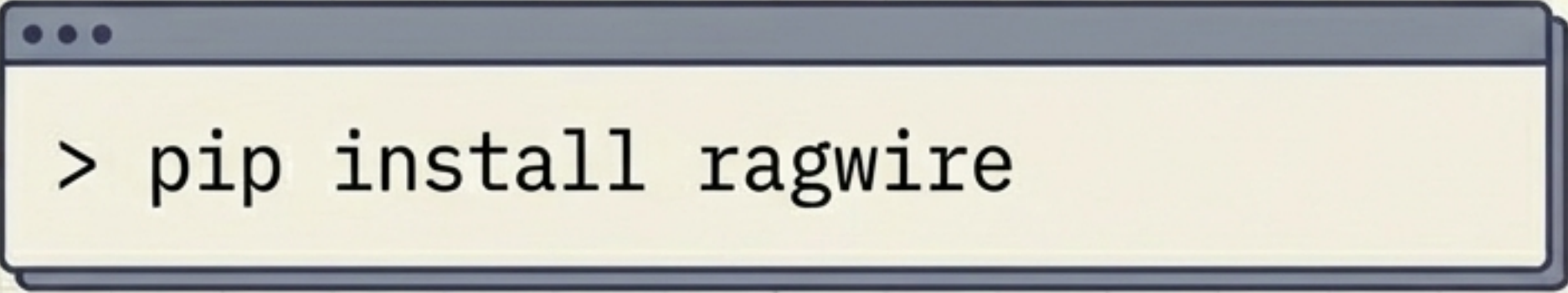
| Provider      | Embeddings | LLM | Access Tier |
|---------------|------------|-----|-------------|
| Ollama        | ✓          | ✓   | 🖥️          |
| OpenAI        | ✓          | ✓   | ☁️          |
| Google Gemini | ✓          | ✓   | ☁️          |
| Groq          | —          | ✓   | ☁️          |
| Anthropic     | —          | ✓   | ☁️          |
| HuggingFace   | ✓          | —   | 🖥️          |
| FastEmbed     | ✓          | —   | 🖥️          |

# The Complete RAG Architecture

Hybrid search, automated metadata extraction, and zero-duplicate ingestion—wired together and ready for production.



# Deploy RAGWire Today



```
> pip install ragwire
```

A terminal window with a dark blue header and a light beige body. It contains the command `> pip install ragwire`. An orange arrow points from the first QR code to the terminal window.



Package  
[pypi.org/project/ragwire](https://pypi.org/project/ragwire)



Repository  
[github.com/laxmimerit/RAGWire](https://github.com/laxmimerit/RAGWire)



Tutorials  
[youtube.com/kgptalkie](https://youtube.com/kgptalkie)